

# Image Interpolation — From Scratch

**Implemented in NumPy (No OpenCV)**

**Author:** Chi Dai

**Team:** CVVN

## Objective

Explain and implement **image interpolation** methods:

- Nearest Neighbor
- Bilinear
- Bicubic
- (Optionally Edge-Directed)

We'll understand:

1. What interpolation means
2. How each method computes new pixels
3. How to implement from scratch

## What Is Image Interpolation?

When we **resize** an image (e.g., 2× upsample),  
we need to estimate **new pixels** that weren't in the original image.

For every pixel in the **new image**,  
we map it back to a **non-integer coordinate** in the old image:

$$x = \frac{x'}{s}, \quad y = \frac{y'}{s}$$

Then, use nearby pixels to **estimate** its color.

## Why It Matters

| Method   | Accuracy | Speed                                                                                      | Smoothness                                                                                      | Typical Use          |
|----------|----------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|----------------------|
| Nearest  | Low      |  Fast     | ✖ Blocky                                                                                        | quick preview        |
| Bilinear | Medium   |  Moderate |  Smooth      | scaling UI, basic SR |
| Bicubic  | High     |  Slow     |  Very smooth | high-quality resize  |



## Common Idea

All methods compute the output pixel as a **weighted sum** of neighboring pixels in the original image.

$$I'(x', y') = \sum_{i,j} w(i, j) \cdot I(x + i, y + j)$$

Weights (  $w(i,j)$  ) depend on **distance** between the new pixel and the old ones.

## Nearest Neighbor — Concept

Pick the **closest** pixel and copy its value.

$$I'(x', y') = I(\text{round}(x), \text{round}(y))$$

 Fast

 Blocky edges

## Nearest Implementation

```
def nearest_interpolation(img, x, y):  
    h, w, c = img.shape  
    x0 = np.clip(np.round(x).astype(int), 0, w - 1)  
    y0 = np.clip(np.round(y).astype(int), 0, h - 1)  
    return img[y0, x0]
```

## Bilinear Interpolation — Concept

Use the **4 nearest neighbors** to estimate pixel color by **linear weighting** both horizontally and vertically.

$$I'(x', y') = (1 - dx)(1 - dy)I_{00} + dx(1 - dy)I_{10} + (1 - dx)dyI_{01} + dxdyI_{11}$$

 Smooth transition

 May blur fine details



## Bilinear Implementation

```
def bilinear_interpolation(img, x, y):  
    h, w, c = img.shape  
    x0 = np.floor(x).astype(int)  
    x1 = np.clip(x0 + 1, 0, w - 1)  
    y0 = np.floor(y).astype(int)  
    y1 = np.clip(y0 + 1, 0, h - 1)  
  
    dx = x - x0  
    dy = y - y0  
  
    Ia = img[y0, x0]  
    Ib = img[y1, x0]  
    Ic = img[y0, x1]  
    Id = img[y1, x1]  
  
    wa = (1 - dx) * (1 - dy)  
    wb = (1 - dx) * dy  
    wc = dx * (1 - dy)  
    wd = dx * dy  
  
    return (Ia*wa[..., None] + Ib*wb[..., None] +  
            Ic*wc[..., None] + Id*wd[..., None])
```

## Bicubic Interpolation — Concept

Uses **16 neighbors** (4×4 grid)  
and fits a **cubic polynomial surface**.

$$I'(x', y') = \sum_{i=-1}^2 \sum_{j=-1}^2 I(x + i, y + j) \cdot w(i) \cdot w(j)$$

- ✓ Very smooth
- ✓ Preserves edges better
- ✗ Computationally expensive



## Cubic Weight Function

Common choice: **Catmull-Rom spline**

$$w(t) = \begin{cases} (a + 2)|t|^3 - (a + 3)|t|^2 + 1, & |t| \leq 1 \\ a|t|^3 - 5a|t|^2 + 8a|t| - 4a, & 1 < |t| < 2 \\ 0, & |t| \geq 2 \end{cases}$$

Usually (  $a = -0.5$  ).

## Bicubic Implementation

```
def cubic_weight(t):  
    a = -0.5  
    abs_t = np.abs(t)  
    abs_t2 = abs_t**2  
    abs_t3 = abs_t**3  
    w = np.zeros_like(t)  
  
    mask1 = abs_t <= 1  
    mask2 = (abs_t > 1) & (abs_t < 2)  
  
    w[mask1] = (a+2)*abs_t3[mask1] - (a+3)*abs_t2[mask1] + 1  
    w[mask2] = a*abs_t3[mask2] - 5*a*abs_t2[mask2] + 8*a*abs_t[mask2] - 4*a  
    return w
```

## Bicubic Main Loop

```
def bicubic_interpolation(img, x, y):
    h, w, c = img.shape
    out = np.zeros((x.shape[0], x.shape[1], c), dtype=np.float32)
    x0 = np.floor(x).astype(int)
    y0 = np.floor(y).astype(int)

    for j in range(-1, 3):
        for i in range(-1, 3):
            xi = np.clip(x0 + i, 0, w - 1)
            yj = np.clip(y0 + j, 0, h - 1)
            wx = cubic_weight(x - xi)
            wy = cubic_weight(y - yj)
            wxy = wx * wy
            out += img[yj, xi] * wxy[..., None]
    return out
```

## Summary of Differences

| Method   | Neighbors | Formula Type | Look        | Use Case        |
|----------|-----------|--------------|-------------|-----------------|
| Nearest  | 1         | Constant     | Blocky      | Fast preview    |
| Bilinear | 4         | Linear       | Smooth      | Simple scaling  |
| Bicubic  | 16        | Cubic        | Very smooth | High-quality SR |



## Putting It All Together

```
def sr_interpolation(img: Image.Image, method='nearest', scale=2):
    img = np.array(img).astype(np.float32)
    h, w, c = img.shape
    new_h, new_w = int(h * scale), int(w * scale)

    y = np.arange(new_h) / scale
    x = np.arange(new_w) / scale
    y, x = np.meshgrid(y, x)

    if method == 'nearest':
        out = nearest_interpolation(img, x, y)
    elif method == 'bilinear':
        out = bilinear_interpolation(img, x, y)
    elif method == 'bicubic':
        out = bicubic_interpolation(img, x, y)

    return Image.fromarray(np.clip(out, 0, 255).astype(np.uint8))
```

## Takeaway

- Interpolation is **key** in resizing, SR, and warping.
- Implementing it manually builds deep intuition.
- Bicubic offers the best visual quality trade-off.

Next step → **Edge-Directed Interpolation** (adaptive by gradient direction).



 **Thank You**

 Presented by: Chi Dai

 Source code: In `interpolation` branch